Traffic Signal RL Project
Complete Code and Results

Generated: 2026-03-19 15:14:04

Reinforcement Learning for Traffic Optimization

Configuration File
config.json

```
{
    "project": {
        "name": "Traffic Signal RL",
        "version": "3.0",
        "date": "2024"
    },
    "data": {
        "n\_days": 30,
        "n\_intersections": 5,
        "base\_arrival": 8,
        "random\_seed": 42
    },
    "agent": {
        "alpha": 0.3,
        "gamma": 0.95,
        "epsilon\_start": 1.0,
        "epsilon\_min": 0.01,
        "epsilon\_decay": 0.995
    },
    "environment": {
        "n\_queue\_bins": 10,
        "n\_hour\_bins": 6,
        "max\_steps\_per\_episode": 50,
        "max\_queue": 50,
        "discharge\_rate": 0.5,
        "queue\_clamp\_min": 5,
        "queue\_clamp\_max": 25,
        "reward\_scale": 10.0,
        "green\_times": [
            10,
            15,
            20,
            25,
            30,
            35,
            40,
            45,
            50,
            55
        ]
    },
    "training": {
        "n\_episodes": 2000,
        "log\_interval": 50,
        "test\_interval": 100,
        "test\_episodes": 10,
        "save\_model": true
    },
    "ml\_models": {
        "train": true,
        "test\_size": 0.2,
        "random\_forest": {
            "n\_estimators": 100
        },
        "svr": {
            "C": 10,
            "epsilon": 0.5
        }
    },
    "evaluation": {
        "n\_episodes": 100,
        "fixed\_timer\_action": 4
    },
    "visualization": {
        "smoothing\_window": 50,
        "figure\_dpi": 150,
        "style": "whitegrid"
    },
    "reporting": {
        "generate\_pdf": true,
        "include\_charts": true,
        "save\_metrics\_csv": true
    }
}
```

Git Ignore
.gitignore

```
# Virtual Environment
venv/
.venv/
env/
ENV/

# Python cache
\_\_pycache\_\_/
*.pyc
*.pyo
*.pyd
.pytest\_cache/

# Logs and generated data (but keep the folders)
logs/*.json
logs/*.npy
!logs/.gitkeep

# Models (too large for GitHub)
models/*.npy
models/*.pkl
!models/.gitkeep

# Reports (optional - can keep if small)
reports/*.csv
reports/*.txt
!reports/.gitkeep

# Results/charts (keep these - they're small PNGs)
# results/ - we DO want to keep these for presentation

# VS Code settings
.vscode/
*.code-workspace

# Windows
Thumbs.db
desktop.ini
.DS\_Store
```

Requirements
requirements.txt

```
numpy==1.26.4
pandas==2.2.2
matplotlib==3.9.0
seaborn==0.13.2
scikit-learn==1.5.0
joblib==1.4.2
tqdm==4.66.4
pyyaml
fpdf2
```

Main Pipeline
main.py

```python
"""
FILE: main.py
PURPOSE: Run full pipeline with config, reporting, and automatic cleanup
"""

import argparse
import json
from pathlib import Path

from data.generate\_synthetic\_data import generate\_data
from src.train import train
from src.evaluate import evaluate
from src.ml\_models import train\_all\_models
from src.visualize import (
    plot\_learning\_curve,
    plot\_step\_by\_step,
    plot\_evaluation\_comparison,
    plot\_ml\_comparison,
)
from src.config import load\_config, print\_config\_summary
from src.report import generate\_report


def cleanup\_reports(keep\_last=1):
    """Delete old report files, keeping only the most recent ones."""
    reports\_dir = Path("reports")
    if not reports\_dir.exists():
        return

    print("\n[CLEAN] Cleaning up old reports...")

    # Get all report files
    report\_files = list(reports\_dir.glob("report\_*.txt"))
    csv\_files = list(reports\_dir.glob("*.csv"))
    all\_files = report\_files + csv\_files

    if not all\_files:
        print("   No old reports to clean")
        return

    # Sort by creation time (newest first)
    all\_files.sort(key=lambda x: x.stat().st\_ctime, reverse=True)

    # Keep the newest 'keep\_last' files, delete the rest
    files\_to\_delete = all\_files[keep\_last:]

    for f in files\_to\_delete:
        try:
            f.unlink()
            print(f"   Deleted: {f.name}")
        except Exception as e:
            print(f"   Could not delete {f.name}: {e}")

    print(f"   Kept {len(all\_files[:keep\_last])} most recent files")


def main():
    parser = argparse.ArgumentParser(description="Traffic RL Project")

    # Basic arguments
    parser.add\_argument(
        "--episodes", type=int, default=None, help="Override number of episodes"
    )
    parser.add\_argument("--skip-data", action="store\_true", help="Skip data generation")
    parser.add\_argument("--skip-ml", action="store\_true", help="Skip ML model training")
    parser.add\_argument(
        "--report", action="store\_true", help="Generate report after run"
    )
    parser.add\_argument(
        "--config", type=str, default="config.json", help="Path to config file"
    )
    parser.add\_argument(
        "--quick-test",
        action="store\_true",
        help="Run quick test (500 episodes, skip ML)",
    )

```

Data Package Init
data/__init__.py

1 |

Synthetic Data Generator
data/generate_synthetic_data.py

```python
import numpy as np
import pandas as pd
from pathlib import Path

np.random.seed(42)


def get\_traffic\_multiplier(hour: int) -> float:
    """Traffic intensity multiplier based on hour of day."""
    if 7 <= hour <= 9:
        return np.random.uniform(1.8, 2.5)  # Morning peak
    elif 17 <= hour <= 19:
        return np.random.uniform(2.0, 3.0)  # Evening peak
    elif 11 <= hour <= 14:
        return np.random.uniform(1.2, 1.6)  # Lunch
    elif 0 <= hour <= 5:
        return np.random.uniform(0.1, 0.3)  # Night
    else:
        return np.random.uniform(0.6, 1.2)  # Normal


def generate\_data(
    n\_days=30, n\_intersections=4, output\_path="data/traffic\_data.csv"
) -> pd.DataFrame:
    records = []
    base\_arrival = 8

    for day in range(n\_days):
        is\_weekend = day % 7 >= 5
        weekend\_factor = 0.6 if is\_weekend else 1.0
        for hour in range(24):
            for minute in range(0, 60, 5):
                for iid in range(n\_intersections):
                    mult = get\_traffic\_multiplier(hour)
                    ifact = np.random.uniform(0.7, 1.3)
                    arr = max(
                        0,
                        base\_arrival * mult * weekend\_factor * ifact
                        + np.random.normal(0, 0.5),
                    )
                    green = np.random.randint(10, 61)
                    thru = green * 0.5
                    queue = max(0, arr * 5 - thru + np.random.normal(0, 1))
                    wait = max(0, (queue / max(1, thru)) * 30 + np.random.normal(0, 2))
                    records.append(
                        {
                            "day": day,
                            "hour": hour,
                            "minute": minute,
                            "intersection\_id": iid,
                            "is\_weekend": int(is\_weekend),
                            "arrival\_rate": round(arr, 2),
                            "green\_time": green,
                            "queue\_length": round(queue, 2),
                            "waiting\_time": round(wait, 2),
                            "time\_of\_day": f"{hour:02d}:{minute:02d}",
                        }
                    )

    df = pd.DataFrame(records)
    Path(output\_path).parent.mkdir(parents=True, exist\_ok=True)
    df.to\_csv(output\_path, index=False)
    print(f"Generated {len(df):,} rows -> {output\_path}")
    return df


if \_\_name\_\_ == "\_\_main\_\_":
    df = generate\_data()
    print(df.head())
    print(df.describe())

```

Source Package Init
src/__init__.py

1 |

Configuration Loader
src/config.py

```
"""
FILE: src/config.py
PURPOSE: Load and validate configuration from JSON
UPDATED: Now includes ALL configuration sections
"""

import json
from pathlib import Path


def load\_config(config\_path="config.json"):
    """Load configuration from JSON file."""
    try:
        with open(config\_path, "r") as f:
            config = json.load(f)
        print(f"[OK] Loaded config from {config\_path}")
        return config
    except FileNotFoundError:
        print(f"[WARN]? Config file not found. Using defaults.")
        return get\_default\_config()
    except json.JSONDecodeError as e:
        print(f"[ERROR] Error parsing config: {e}")
        return get\_default\_config()


def get\_default\_config():
    """Return default configuration with ALL sections."""
    return {
        "project": {"name": "Traffic Signal RL", "version": "3.0", "random\_seed": 42},
        "data": {"n\_days": 30, "n\_intersections": 5, "base\_arrival": 8},
        "agent": {
            "alpha": 0.3,
            "gamma": 0.95,
            "epsilon\_start": 1.0,
            "epsilon\_min": 0.01,
            "epsilon\_decay": 0.995,
        },
        "environment": {
            "n\_queue\_bins": 10,
            "n\_hour\_bins": 6,
            "max\_steps\_per\_episode": 50,
            "max\_queue": 50,
            "discharge\_rate": 0.5,
            "queue\_clamp\_min": 5,
            "queue\_clamp\_max": 25,
            "reward\_scale": 10.0,
            "green\_times": [10, 15, 20, 25, 30, 35, 40, 45, 50, 55],
        },
        "training": {
            "n\_episodes": 1000,
            "log\_interval": 50,
            "test\_interval": 100,
            "test\_episodes": 10,
            "save\_model": True,
        },
        "ml\_models": {
            "train": True,
            "test\_size": 0.2,
            "random\_forest": {"n\_estimators": 100},
            "svr": {"C": 10, "epsilon": 0.5},
        },
        "evaluation": {"n\_episodes": 100, "max\_steps": 50, "fixed\_timer\_action": 2},
        "visualization": {
            "smoothing\_window": 50,
            "figure\_dpi": 150,
            "style": "whitegrid",
        },
        "reporting": {
            "generate\_pdf": True,
            "include\_charts": True,
            "save\_metrics\_csv": True,
        },
    }


def get\_config\_section(config, section, default=None):
    """Safely get a section from config with default fallback."""
```

Config Cache
src/config_loader.py

```
"""
FILE: src/config\_loader.py
PURPOSE: Single source for config loading to avoid repeated file reads
"""

import json
from pathlib import Path
from src.config import load\_config as load\_config\_full

# Global config cache
\_CONFIG\_CACHE = None


def get\_config():
    """Get config singleton (loads once, caches for future calls)."""
    global \_CONFIG\_CACHE
    if \_CONFIG\_CACHE is None:
        \_CONFIG\_CACHE = load\_config\_full()
    return \_CONFIG\_CACHE


def get\_section(section, default=None):
    """Get a specific section from config."""
    config = get\_config()
    return config.get(section, default or {})


def get\_value(section, key, default=None):
    """Get a specific value from config."""
    config = get\_config()
    try:
        return config[section][key]
    except (KeyError, TypeError):
        return default

```

Traffic Environment
src/environment.py

```python
"""
FILE: src/environment.py
PURPOSE: Simulate a traffic intersection with configurable parameters
"""

import numpy as np
import pandas as pd
from src.config\_loader import get\_section, get\_value


class TrafficEnvironment:
    """
    Simulates a traffic intersection.

    State:   (queue\_north\_bin, queue\_south\_bin, hour\_bin) -> single int
    Size:    n\_queue\_bins^2 * n\_hour\_bins states
    Actions: n\_actions green-time choices from config
    Reward:  -(queue\_north + queue\_south) / reward\_scale
    """

    # Load environment config once
    ENV\_CONFIG = get\_section("environment", {})

    # Get green times from config or use defaults
    GREEN\_TIMES = ENV\_CONFIG.get(
        "green\_times", [10, 15, 20, 25, 30, 35, 40, 45, 50, 55]
    )
    N\_ACTIONS = len(GREEN\_TIMES)

    def \_\_init\_\_(self, data\_path: str = "data/traffic\_data.csv"):
        self.df = pd.read\_csv(data\_path)

        # Load configuration
        self.n\_queue\_bins = self.ENV\_CONFIG.get("n\_queue\_bins", 10)
        self.n\_hour\_bins = self.ENV\_CONFIG.get("n\_hour\_bins", 6)
        self.max\_queue = self.ENV\_CONFIG.get("max\_queue", 30)
        self.queue\_clamp\_min = self.ENV\_CONFIG.get("queue\_clamp\_min", 5)
        self.queue\_clamp\_max = self.ENV\_CONFIG.get("queue\_clamp\_max", 25)
        self.reward\_scale = self.ENV\_CONFIG.get("reward\_scale", 10.0)

        # State size calculation
        self.state\_size = self.n\_queue\_bins**2 * self.n\_hour\_bins

        # Cache queue bins once for performance
        self.\_bins = np.linspace(0, self.max\_queue, self.n\_queue\_bins + 1)

        # Initialize state variables
        self.queue\_north = 10.0
        self.queue\_south = 10.0
        self.hour = 8
        self.reset()

    def \_hour\_bin(self, hour: int) -> int:
        """Convert hour to discrete bin based on time-of-day patterns."""
        if hour < 6:
            return 0
        elif hour < 7:
            return 1
        elif hour < 10:
            return 2
        elif hour < 16:
            return 3
        elif hour < 19:
            return 4
        else:
            return 5

    def \_queue\_bin(self, q: float) -> int:
        """Convert queue length to discrete bin."""
        bin\_idx = int(np.digitize(q, self.\_bins)) - 1
        return min(bin\_idx, self.n\_queue\_bins - 1)

    def \_encode(self, qn: float, qs: float, hour: int) -> int:
        """Encode state as single integer."""
        bn = self.\_queue\_bin(qn)
        bs = self.\_queue\_bin(qs)
        bh = self.\_hour\_bin(hour)
```

Q-Learning Agent
src/agent.py

```python
"""
FILE: src/agent.py
PURPOSE: Q-Learning agent with epsilon-greedy action selection
"""

import numpy as np
import json
from pathlib import Path
from src.config\_loader import get\_section


class QLearningAgent:
    """
    Q-Learning agent with epsilon-greedy action selection.
    All hyperparameters can be set in config.json under "agent" section.
    """

    # Load agent config once
    AGENT\_CONFIG = get\_section("agent", {})

    def \_\_init\_\_(
        self,
        state\_size: int,
        n\_actions: int,
        alpha: float = None,
        gamma: float = None,
        epsilon: float = None,
        epsilon\_min: float = None,
        epsilon\_decay: float = None,
    ):
        """
        Initialize Q-Learning agent with values from config.
        """
        # Use provided values, otherwise use config, otherwise use defaults
        self.alpha = alpha if alpha is not None else self.AGENT\_CONFIG.get("alpha", 0.3)
        self.gamma = (
            gamma if gamma is not None else self.AGENT\_CONFIG.get("gamma", 0.95)
        )
        self.epsilon = (
            epsilon
            if epsilon is not None
            else self.AGENT\_CONFIG.get("epsilon\_start", 1.0)
        )
        self.epsilon\_min = (
            epsilon\_min
            if epsilon\_min is not None
            else self.AGENT\_CONFIG.get("epsilon\_min", 0.01)
        )
        self.epsilon\_decay = (
            epsilon\_decay
            if epsilon\_decay is not None
            else self.AGENT\_CONFIG.get("epsilon\_decay", 0.995)
        )

        self.q\_table = np.zeros((state\_size, n\_actions))

    def choose\_action(self, state: int) -> int:
        """Epsilon-greedy action selection."""
        if np.random.random() < self.epsilon:
            return np.random.randint(self.q\_table.shape[1])
        return int(np.argmax(self.q\_table[state]))

    def update(self, s: int, a: int, r: float, s\_next: int, done: bool):
        """Update Q-value using Q-learning rule."""
        best\_next = 0.0 if done else float(np.max(self.q\_table[s\_next]))
        td\_target = r + self.gamma * best\_next
        td\_error = td\_target - self.q\_table[s, a]
        self.q\_table[s, a] += self.alpha * td\_error

    def decay\_epsilon(self):
        """Reduce exploration rate over time."""
        self.epsilon = max(self.epsilon\_min, self.epsilon * self.epsilon\_decay)

    def save(self, path: str = "models/q\_table.npy"):
        """Save Q-table and metadata."""
        Path(path).parent.mkdir(parents=True, exist\_ok=True)
        np.save(path, self.q\_table)
```

Training Loop
src/train.py

```
import numpy as np
import json
from pathlib import Path
from tqdm import tqdm
from src.environment import TrafficEnvironment
from src.agent import QLearningAgent


def train(
    n\_episodes=1000, max\_steps=50, data\_path="data/traffic\_data.csv", log\_interval=50
):

    env = TrafficEnvironment(data\_path)
    agent = QLearningAgent(env.state\_size, env.N\_ACTIONS)
    history = {"episode\_rewards": [], "epsilon\_values": [], "mean\_queue\_per\_ep": []}

    print(f"Training {n\_episodes} episodes ...")
    for ep in tqdm(range(n\_episodes)):
        state = env.reset()
        total\_rew = total\_q = steps = 0

        for \_ in range(max\_steps):
            action = agent.choose\_action(state)
            next\_s, rew, done = env.step(action)
            agent.update(state, action, rew, next\_s, done)
            state = next\_s
            total\_rew += rew
            total\_q += env.queue\_north + env.queue\_south
            steps += 1
            if done:
                break

        agent.decay\_epsilon()
        history["episode\_rewards"].append(total\_rew)
        history["epsilon\_values"].append(agent.epsilon)
        history["mean\_queue\_per\_ep"].append(total\_q / steps)

        if (ep + 1) % log\_interval == 0:
            recent = np.mean(history["episode\_rewards"][-log\_interval:])
            print(
                f"  Ep {ep+1:>5} | Avg Reward: {recent:>8.2f} | Eps:{agent.epsilon:.4f}"
            )

    agent.save()
    log\_path = Path("logs/training\_history.json")
    log\_path.parent.mkdir(exist\_ok=True)
    log\_path.write\_text(json.dumps(history))
    print("Training complete. Logs saved.")

    # Run quick evaluation and save results
    try:
        print("\n[CHART] Running quick evaluation...")
        from src.evaluate import evaluate

        eval\_results = evaluate(n\_episodes=50)

        # Save evaluation results
        eval\_path = Path("logs/evaluation\_results.json")

        # Convert numpy values to Python floats for JSON
        clean\_results = {}
        for agent\_name, metrics in eval\_results.items():
            clean\_results[agent\_name] = {
                "mean\_reward": float(metrics["mean\_reward"]),
                "std\_reward": float(metrics["std\_reward"]),
                "mean\_queue": float(metrics["mean\_queue"]),
                "std\_queue": float(metrics["std\_queue"]),
            }

        eval\_path.write\_text(json.dumps(clean\_results, indent=2))
        print(f"[OK] Evaluation results saved to {eval\_path}")
    except Exception as e:
        print(f"[WARN]? Could not run evaluation: {e}")

    return history

```

Evaluation
src/evaluate.py

```python
"""
FILE: src/evaluate.py
PURPOSE: Compare RL agent vs fixed timer vs random agent
"""

import numpy as np
from src.environment import TrafficEnvironment
from src.agent import QLearningAgent
from src.config\_loader import get\_section


# Load evaluation config
EVAL\_CONFIG = get\_section("evaluation", {})


def run\_policy(env, policy\_fn, n\_episodes=None, max\_steps=None):
    """Run a policy for multiple episodes and collect stats."""
    if n\_episodes is None:
        n\_episodes = EVAL\_CONFIG.get("n\_episodes", 100)
    if max\_steps is None:
        max\_steps = EVAL\_CONFIG.get("max\_steps", 50)

    total\_rewards, total\_queues = [], []

    for \_ in range(n\_episodes):
        state = env.reset()
        episode\_reward = 0
        episode\_queue = 0
        steps = 0

        for \_ in range(max\_steps):
            action = policy\_fn(state)
            state, reward, done = env.step(action)
            episode\_reward += reward
            episode\_queue += env.queue\_north + env.queue\_south
            steps += 1
            if done:
                break

        total\_rewards.append(episode\_reward)
        total\_queues.append(episode\_queue / steps)

    return {
        "mean\_reward": float(np.mean(total\_rewards)),
        "std\_reward": float(np.std(total\_rewards)),
        "mean\_queue": float(np.mean(total\_queues)),
        "std\_queue": float(np.std(total\_queues)),
    }


def evaluate(data\_path="data/traffic\_data.csv", n\_episodes=None):
    """Compare RL agent vs fixed timer vs random agent."""

    if n\_episodes is None:
        n\_episodes = EVAL\_CONFIG.get("n\_episodes", 100)

    max\_steps = EVAL\_CONFIG.get("max\_steps", 50)
    fixed\_action = EVAL\_CONFIG.get("fixed\_timer\_action", 2)

    print("\n" + "=" * 60)
    print("EVALUATION - Comparing Controllers")
    print("=" * 60)
    print(f"Episodes: {n\_episodes} | Max steps: {max\_steps}")

    env = TrafficEnvironment(data\_path)

    # RL Agent
    agent = QLearningAgent(env.state\_size, env.N\_ACTIONS)
    try:
        agent.load()
        agent.epsilon = 0.0
        print("[OK] RL Agent loaded")
    except:
        print("[WARN] No trained model found, using untrained agent")

    rl\_results = run\_policy(
        env, lambda s: agent.choose\_action(s), n\_episodes, max\_steps
```

ML Models Comparison
src/ml_models.py

```
import numpy as np
import pandas as pd
import joblib
from pathlib import Path
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear\_model import LinearRegression
from sklearn.svm import SVR
from sklearn.model\_selection import train\_test\_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean\_absolute\_error, r2\_score

FEATURES = ["arrival\_rate", "queue\_length", "hour", "is\_weekend"]
TARGET = "green\_time"


def train\_all\_models(data\_path="data/traffic\_data.csv"):
    df = pd.read\_csv(data\_path)
    X = df[FEATURES]
    y = df[TARGET]
    X\_tr, X\_te, y\_tr, y\_te = train\_test\_split(X, y, test\_size=0.2, random\_state=42)

    scaler = StandardScaler()
    X\_tr\_s = scaler.fit\_transform(X\_tr)
    X\_te\_s = scaler.transform(X\_te)

    models = {
        "RandomForest": RandomForestRegressor(n\_estimators=100, random\_state=42),
        "LinearRegression": LinearRegression(),
        "SVR": SVR(kernel="rbf", C=10, epsilon=0.5),
    }
    results = {}
    Path("models").mkdir(exist\_ok=True)

    for name, model in models.items():
        print(f"Training {name}...")
        scaled = name in ["LinearRegression", "SVR"]
        model.fit(X\_tr\_s if scaled else X\_tr, y\_tr)
        preds = model.predict(X\_te\_s if scaled else X\_te)
        mae = mean\_absolute\_error(y\_te, preds)
        r2 = r2\_score(y\_te, preds)
        results[name] = {"mae": round(mae, 3), "r2": round(r2, 3)}
        print(f"  {name}: MAE={mae:.3f}  R2={r2:.3f}")
        joblib.dump(model, f"models/{name.lower()}.pkl")

    return results

```

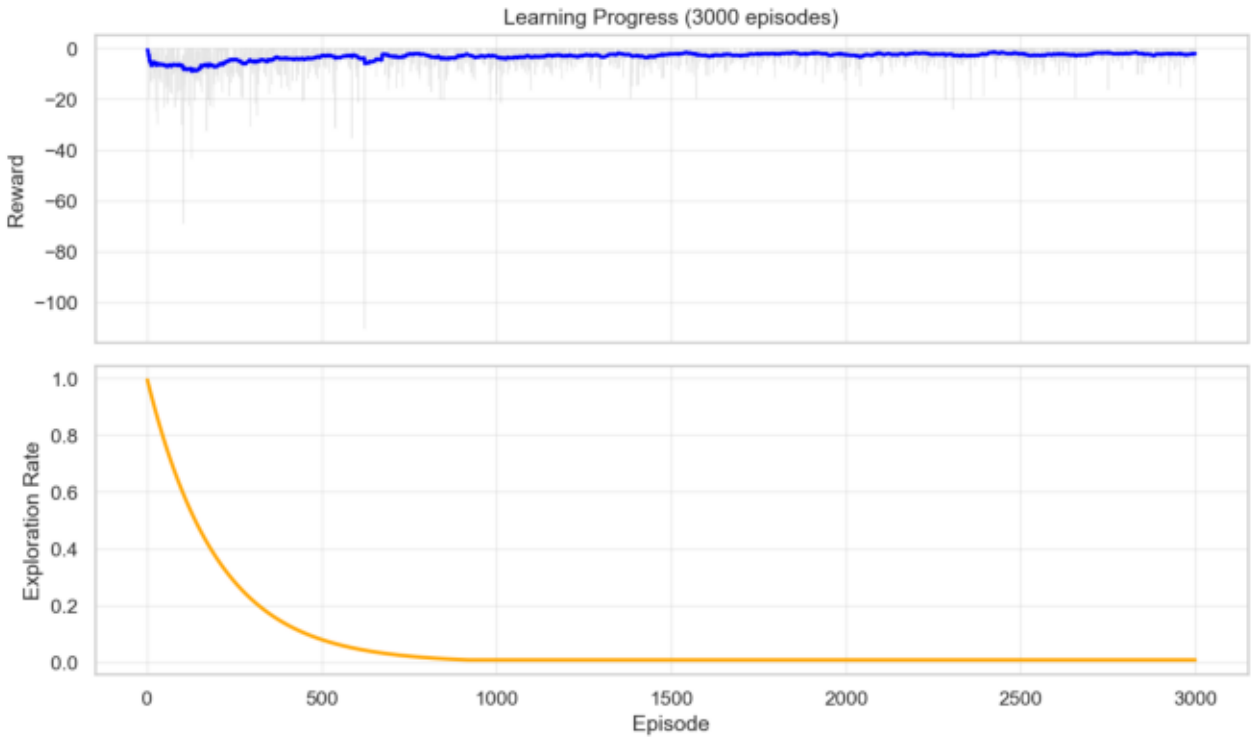
Visualization
src/visualize.py

```python
"""
FILE: src/visualize.py
PURPOSE: Create clean, presentation-ready charts from training logs
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
from pathlib import Path

# Create results directory
Path("results").mkdir(exist\_ok=True)
sns.set\_theme(style="whitegrid", palette="muted")


def load\_training\_history(history\_path="logs/training\_history.json"):
    """Load training history with error handling."""
    try:
        with open(history\_path, "r") as f:
            history = json.load(f)
        n\_episodes = len(history.get("episode\_rewards", []))
        print(f"   [OK] Loaded {n\_episodes} episodes")
        return history
    except:
        print(f"   [WARN] No training history found")
        return None


def plot\_learning\_curve(history\_path="logs/training\_history.json"):
    """Plot training progress."""
    history = load\_training\_history(history\_path)
    if not history:
        return

    rewards = history["episode\_rewards"]
    eps = history["epsilon\_values"]
    n\_episodes = len(rewards)

    # Smooth rewards
    window = min(50, n\_episodes // 10)
    smooth = pd.Series(rewards).rolling(window, min\_periods=1).mean()

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

    # Plot 1: Learning curve
    ax1.plot(rewards, alpha=0.2, color="gray", linewidth=0.5)
    ax1.plot(smooth, color="blue", lw=2)
    ax1.set\_ylabel("Reward")
    ax1.set\_title(f"Learning Progress ({n\_episodes} episodes)")
    ax1.grid(True, alpha=0.3)

    # Plot 2: Exploration decay
    ax2.plot(eps, color="orange", lw=2)
    ax2.set\_xlabel("Episode")
    ax2.set\_ylabel("Exploration Rate")
    ax2.grid(True, alpha=0.3)

    plt.tight\_layout()
    plt.savefig("results/learning\_curve.png", dpi=150, bbox\_inches="tight")
    plt.close()
    print("   [OK] Saved: results/learning\_curve.png")


def plot\_step\_by\_step(history\_path="logs/training\_history.json", steps=5):
    """Show learning snapshots at different stages."""
    history = load\_training\_history(history\_path)
    if not history:
        return

    rewards = history["episode\_rewards"]
    n\_episodes = len(rewards)

    # Pick checkpoints
    checkpoints = np.linspace(0, n\_episodes - 1, steps + 1, dtype=int)[1:]

    fig, axes = plt.subplots(1, steps, figsize=(15, 3))
```

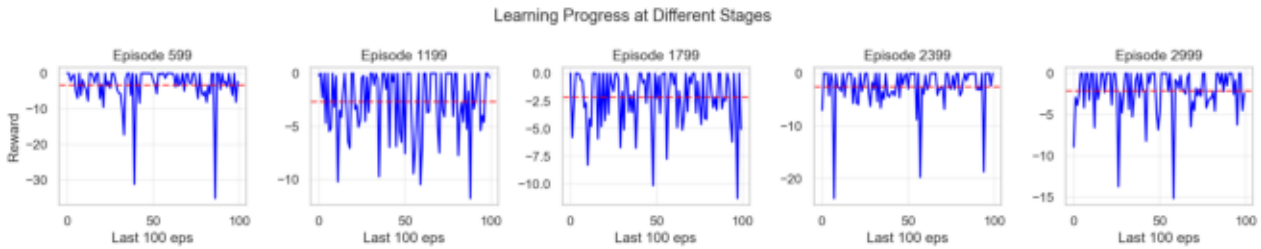
Report Generator
src/report.py

```python
import json
import numpy as np
import pandas as pd
from datetime import datetime
from pathlib import Path
from src.config\_loader import get\_section


class ReportGenerator:
    """Generate reports from training logs and evaluation results."""

    def \_\_init\_\_(self):
        self.timestamp = datetime.now().strftime("%Y%m%d\_%H%M%S")
        self.report\_config = get\_section("reporting", {})
        Path("reports").mkdir(exist\_ok=True)
        print(f"   ReportGenerator initialized, timestamp: {self.timestamp}")

    def load\_data(self):
        """Load all results from logs."""
        print("   Loading data from logs...")

        # Load training history
        history\_path = Path("logs/training\_history.json")
        if history\_path.exists():
            with open(history\_path, "r") as f:
                self.history = json.load(f)
            print(
                f"   [OK] Loaded training history: {len(self.history.get('episode\_rewards', []))} episodes"
            )
        else:
            self.history = None
            print("   [WARN] No training history found")

        # Load evaluation results
        eval\_path = Path("logs/evaluation\_results.json")
        if eval\_path.exists():
            with open(eval\_path, "r") as f:
                self.eval\_results = json.load(f)
            print(f"   [OK] Loaded evaluation results")
        else:
            self.eval\_results = None
            print("   [WARN] No evaluation results found")

        # Load ML metrics
        ml\_path = Path("logs/ml\_metrics.json")
        if ml\_path.exists():
            with open(ml\_path, "r") as f:
                self.ml\_results = json.load(f)
            print(f"   [OK] Loaded ML metrics")
        else:
            self.ml\_results = None

    def calculate\_metrics(self):
        """Calculate key performance metrics."""
        metrics = {}

        if self.history:
            rewards = self.history["episode\_rewards"]
            queues = self.history["mean\_queue\_per\_ep"]

            metrics["training"] = {
                "final\_reward": float(np.mean(rewards[-50:])),
                "final\_queue": float(np.mean(queues[-50:])),
                "best\_reward": float(max(rewards)),
                "best\_queue": float(min(queues)),
                "total\_episodes": len(rewards),
            }
            print(f"   [OK] Calculated training metrics")

        if self.eval\_results:
            fixed\_queue = self.eval\_results["Fixed Timer"]["mean\_queue"]
            rl\_queue = self.eval\_results["RL Agent"]["mean\_queue"]
            random\_queue = self.eval\_results["Random Agent"]["mean\_queue"]

            metrics["evaluation"] = {
                "rl\_queue": rl\_queue,
                "fixed\_queue": fixed\_queue,
```

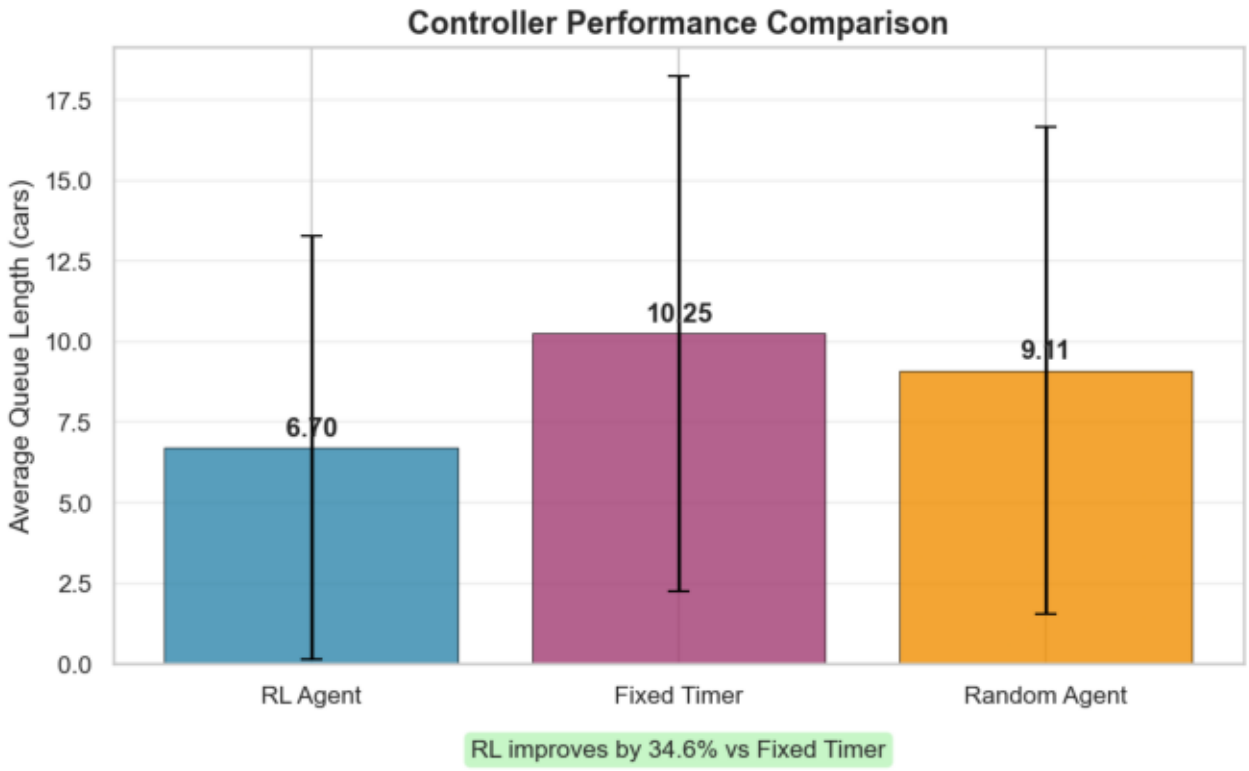
Learning Curve



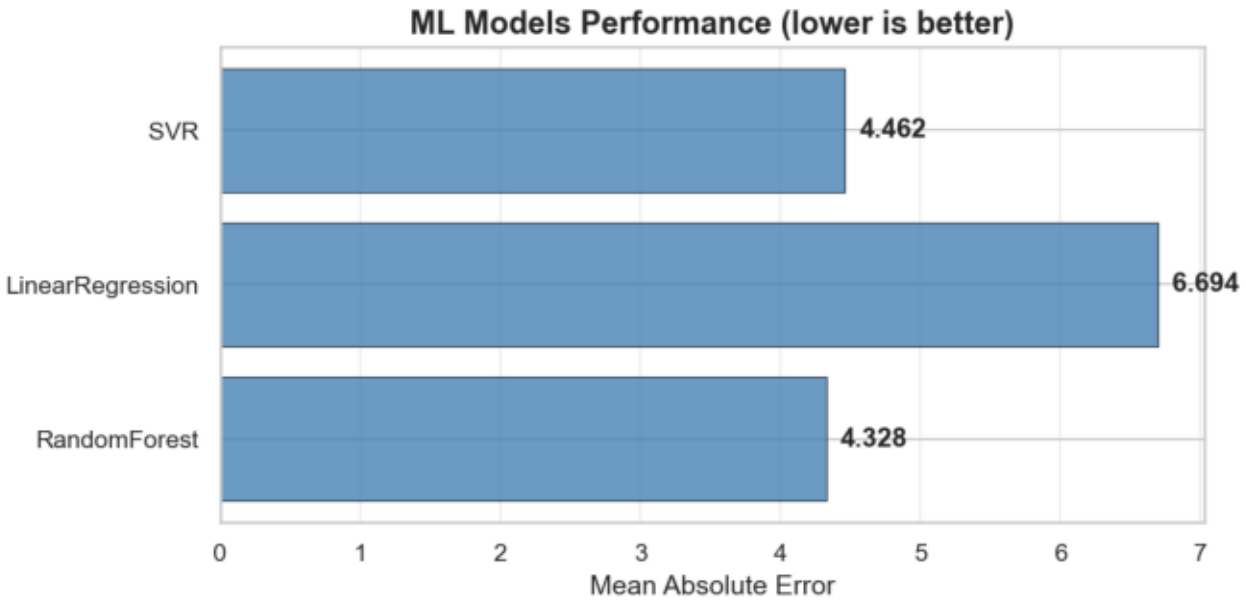
Step-by-Step Learning



Evaluation Comparison



ML Model Comparison



Results Summary

```
================================================================================
TRAFFIC SIGNAL RL PROJECT - COMPLETE REPORT
Generated: 2026-03-19 15:14:04
================================================================================

============================================================
EVALUATION RESULTS
============================================================

RL Agent:
  Mean Reward: -2.190
  Std Reward: 2.568
  Mean Queue: 6.70 cars
  Std Queue: 6.57 cars

Fixed Timer:
  Mean Reward: -5.354
  Std Reward: 4.753
  Mean Queue: 10.25 cars
  Std Queue: 7.98 cars

Random Agent:
  Mean Reward: -4.543
  Std Reward: 4.945
  Mean Queue: 9.11 cars
  Std Queue: 7.57 cars

============================================================
ML MODELS COMPARISON
============================================================

RandomForest:
  MAE: 4.328
  R?: 0.731

LinearRegression:
  MAE: 6.694
  R?: 0.610

SVR:
  MAE: 4.462
  R?: 0.750

============================================================
TRAINING SUMMARY
============================================================

Total Episodes: 3000
Final Avg Reward (last 50): -1.932
Final Avg Queue (last 50): 6.49 cars
Best Reward: 0.000
Best Queue: 0.00 cars

============================================================
KEY RESULT
============================================================

[OK] RL Agent improves by 34.6% over Fixed Timer!
   RL Queue: 6.70 cars
   Fixed Queue: 10.25 cars

================================================================================
```